

Extending R with classes, modules, procs, lambdas, oh my!

Rodrigo Botafogo

Daniel Mossé - University of Pittsburgh

November 19th, 2018 (narrative updated for Galaaz 2.0, 2026)

Contents

1	Introduction	2
2	Bases of Object Programming	3
3	Classes Declaration	3
4	Instance Variables	3
5	Constructor	4
6	Access to Instance Variables (to reach a slot)	5
7	Default Values	7
8	The Empty Object	8
9	To See an Object	8
10	Methods	8
11	To Remove an Object	13
12	Method count_missing	13
13	To See the Methods	14
14	Construction	15
15	Inspector	15
16	The Initializator	19
17	Constructors for Users	23

18 Accessors	24
19 Get	24
20 Set	25
21 The Operator '['	27
22 To Go Further	27
23 Methods Using Several Arguments	28
24 Inheritance	29
25 The 'super' Keyword	31
26 Virtual Classes	34
27 Internal Modification of an Object	37
27.1 Method to Modify a Field	37
28 Conclusions I	38
29 Callbacks and R calling into Ruby	39
30 Conclusions II	39



1 Introduction

This paper introduces and compares Galaaz with R's S4. It is **modeled closely** on "A '(not so)' Short Introduction to S4" by Christophe Genolini and follows the same structure and examples presented there.

Galaaz is a Ruby Gem (library) that allows very tight integration between Ruby and R. Its integration is tighter and more transparent than what one can get between RinRuby or similar solutions in Python, such as Pyper, rpy2 and other similar solutions.

Galaaz 2.0 supports **JRuby** or **CRuby** equally and drives **GNU R** through a **bridge**, so Ruby code can create and manipulate R objects and call R functions while staying idiomatic Ruby. An earlier prototype used Oracle's **GraalVM** with **TruffleRuby** and **FastR**; that stack is historical and is **not** what current Galaaz targets.

2 Bases of Object Programming

In this paper, we will start our discussion from Part II of “The (not so) Short Introduction to S4”, which from now on we will reference as SS4 for “short S4”. Interested readers are directed to this paper to understand the motivation and examples in that paper. In this paper we will present the S4 code from SS4 and then the same code in Ruby/Galaaz. We will not comment on the S4 code, as all the comments can be found in SS4, we will only focus on the Ruby/Galaaz description.

S4 defines classes by using the `setClass` function:

3 Classes Declaration

```
# > setClass(  
# + Class="Trajectories",  
# + representation=representation(  
# + times = "numeric",  
# + traj = "matrix"  
# + )  
# + )
```

4 Instance Variables

In Ruby a class is defined by the keyword ‘class’. Every class should start with a capital letter. S4 ‘slots’ are called ‘instance variables’ in Ruby. Differently from R’s S4, instance variables in Ruby do not have type information. It should be clear though, that S4 type information is also not a “compile” time type, since R is not compiled. The type is checked at runtime. The same checking can be done in Ruby and we will do it later in this document.

In the example below, we create class Trajectories with two instance variables, ‘times’ and ‘matrix’. We will not go over the details of instance variables in Ruby, but here we created those variables with the keyword ‘attr_reader’ and a colon before the variables name:

```
class Trajectories  
  attr_reader :times  
  attr_reader :matrix  
end
```

In order to create a new instance of object Trajectories we call method `new` on the class and we can store the result in a variable (not an instance variable) as below:

```
@traj = Trajectories.new
```

We now have in variable '@traj' a Trajectories object. In Ruby, printing variable 'traj' will only print the class name of the object and not its contents as in R.

```
puts @traj
```

```
## #<RC::Trajectories:0x7df91b21>
```

To see the contents of an object, one needs to access its components using the '.' operator:

```
puts @traj.times
```

5 Constructor

Since there is no content stored in 'times' nor 'matrix', nil is returned. In order to add a value in the variables, we need to add a constructor to the class Trajectories. In R, a constructor is built by default, in Ruby, this has to be created by adding a method called 'initialize'. In the example below, we will create the initializer that accepts two values, a 'times' value and a 'matrix' value and they are used to initialize the value of the instance variables:

```
class Trajectories

  attr_reader :times
  attr_reader :matrix

  #-----
  # Initializes the Trajectories class. Takes two parameters
  # @param times
  # @param matrix
  #-----

  def initialize(times: nil, matrix: nil)
    @times = times
    @matrix = matrix
  end
end
```

Up to this point, everything described is pure Ruby code and has absolutely no relationship with R. We now want to create a Trajectories with a 'times' vector. Ruby has a vector class and we could use this class to create a vector and add it to the 'times' instance variable; however, in order to make use of R's functions, we want to create a R vector to add to 'times'. In Galaaz, creating R objects is done using the corresponding R functions by just preceding them with 'R.', i.e., R functions are all defined in Galaaz in the R namespace.

Since Galaaz is Ruby and not R, some syntax adjustments are sometimes necessary. For instance, in R, a range is represented as '(1:4)', in Ruby, the same range is represented as '(1..4)'. When passing arguments to an R function in R one uses the '=' sign after the slot name; in R, one uses the ':' operator after parameter's name as we can see below:

```
# Create a Trajectories passing a times vector, but no matrix parameter
@traj = Trajectories.new(times: R.c(1, 2, 3, 4))

# Create a Trajectories with times and matrix
@traj2 = Trajectories.new(times: R.c(1, 3), matrix: R.matrix((1..4),
  ncol: 2))
```

6 Access to Instance Variables (to reach a slot)

In order to access data in an instance variable the operator ‘.’ is used. In R, a similar result is obtained by use of the ‘@’ operator, but SS4 does not recommend its use. In Galaaz, the ‘.’ operator is the recommended way of accessing an instance variable.

Now that we have created two trajectories, let’s try to print its instance variables to see that everything is fine:

```
puts @traj.times
```

```
## [1] 1 2 3 4
```

We now have the expected value. Note that the ‘times’ vector is printed exactly as it would if we were using GNU R. Let’s now take a look at variable ‘traj2’:

```
puts @traj2.times
puts
puts @traj2.matrix
```

```
## [1] 1 3
##
##      [,1] [,2]
## [1,]    1    3
## [2,]    2    4
```

Let’s now build the same examples as in SS4: Three hospitals take part in a study. The Pitié Salpêtrière (which has not yet returned its data file, shame on them!), Cochin and Saint-Anne. We first show the code in R and the corresponding Galaaz:

```
> trajPitie <- new(Class="Trajectories")
> trajCochin <- new(
+   Class= "Trajectories",
+   times=c(1,3,4,5),
+   traj=rbind (
+     c(15,15.1, 15.2, 15.2),
+     c(16,15.9, 16,16.4),
+     c(15.2, NA, 15.3, 15.3),
+     c(15.7, 15.6, 15.8, 16)
+   )
+ )
> trajStAnne <- new(
```



```
+   Class= "Trajectories",
+   times=c(1: 10, (6: 16) *2),
+   traj=rbind(
+       matrix (seq (16,19, length=21), ncol=21, nrow=50, byrow=TRUE),
+       matrix (seq (15.8, 18, length=21), ncol=21, nrow=30,
+         byrow=TRUE)
+   )+rnorm (21*80,0,0.2)
+ )
```

This same code in Galaaz becomes:

```
@trajPitie = Trajectories.new

@trajCochin = Trajectories.new(times: R.c(1,3,4,5),
                              matrix: R.rbind(
                                R.c(15,15.1, 15.2, 15.2),
                                R.c(16,15.9, 16,16.4),
                                R.c(15.2, R::NA, 15.3, 15.3),
                                R.c(15.7, 15.6, 15.8, 16)))

@trajStAnne =
  Trajectories.new(
    times: R.c((1..10), R.c(6..16) * 2),
    matrix: (
      R.rbind(
        R.matrix(R.seq(16, 19, length: 21),
                  ncol: 21, nrow: 50, byrow: true),
        R.matrix(R.seq(15.8, 18, length: 21),
                  ncol: 21, nrow: 30, byrow: true)
      ) + R.rnorm(21 * 80, 0, 0.2)
    )
  )
```

Let's check that the 'times' and 'matrix' instance variables were correctly set:

```
puts @trajCochin.times
puts
puts @trajCochin.matrix
puts
puts @trajStAnne.times
```

```
## [1] 1 3 4 5
##
##      [,1] [,2] [,3] [,4]
## g2_v729 15.0 15.1 15.2 15.2
## g2_v730 16.0 15.9 16.0 16.4
## g2_v731 15.2  NA 15.3 15.3
## g2_v732 15.7 15.6 15.8 16.0
##
## [1] 1 2 3 4 5 6 7 8 9 10 12 14 16 18 20 22 24 26 28 30 32
```

We will not at this time print `trajStAnne.matrix`, since this is a huge matrix and the result would just take too much space. Later we will print just a partial view of the matrix.

7 Default Values

Default values are very useful and quite often used in Ruby programs. Although SS4 does not recommend its use, there are many cases in which default values are useful and make code simpler. We have already seen default values in this document, with the default being ‘nil’. This was necessary in order to be able to create our constructor and passing it the proper values.

In the example below, a class TrajectoriesBis is created with default value 1 for times and a matrix with no elements in matrix.

```
class TrajectoriesBis

  attr_reader :times
  attr_reader :matrix

  #-----
  # Initializes the Trajectories class. Takes two parameters
  # @param times
  # @param matrix
  #-----

  def initialize(times: 1, matrix: R.matrix(0))
    @times = times
    @matrix = matrix
  end

end

@traj_bis = TrajectoriesBis.new
```

Let’s take a look at our new class:

```
puts @traj_bis.times
puts
puts @traj_bis.matrix
```

```
## 1
##
##      [,1]
## [1,]      0
```

Note that ‘@traj_bis.times’ is the numeric 1, and what we actually want is a vector with [1] in it.

```
class TrajectoriesBis

  attr_reader :times
  attr_reader :matrix

  #-----
  # Initializes the Trajectories class. Takes two parameters
  # @param times [R::Vector] should be an R vector.
  # @param matrix [R::Matrix] should be an R matrix.
  #-----

end
```

```
# Use R.c to convert number 1 to a vector
def initialize(times: R.c(1), matrix: R.matrix(0))
  @times = times
  @matrix = matrix
end

end

@traj_bis = TrajectoriesBis.new
```

```
puts @traj_bis.times
puts
puts @traj_bis.matrix
```

```
## [1] 1
##
##      [,1]
## [1,]    0
```

8 The Empty Object

When a Trajectories is created with new, and no argument is given, all its instance variables will have the default nil value. Since Ruby has no type information, then there is only one type (or actually no type) of nil. To check if a variable is empty, we check it against the nil value.

9 To See an Object

Ruby has very strong meta-programming features, in particular, one can use introspection to see methods and instance variables from a given class. Method 'instance_variables' shows all the instance variables of an object:

```
puts @traj.instance_variables
```

The description of all meta-programming features of Ruby is well beyond the scope of this document, but it is a very frequent a powerful feature of Ruby, that makes programming in Ruby a different experience than programming in other languages.

10 Methods

Methods are a fundamental feature of object oriented programming. We will now extend our class Trajectories to add methods to it. In SS4, a method 'plot' is added to Trajectories. At this point, Renjin and Galaaz do not yet have plotting capabilities, so we will have to skip this method and go directly to the implementation of the 'print' method.

Below is the R code for method print:

```
> setMethod ("print","Trajectories",
+ function(x,...){
+ cat("*** Class Trajectories, method Print *** \n")
+ cat("* Times ="); print (x@times)
+ cat("* Traj = \n"); print (x@traj)
+ cat("***** End Print (trajectories) ***** \n")
+ }
+ )
```

Now the same code for class Trajectories in Galaaz. In general methods are defined in a class together with all the class definition. We will first use this approach. Later, we will show how to ‘reopen’ a class to add new methods to it.

In this example, we are defining a method named ‘print’. We have been using method ‘puts’ to output data. There is a Ruby method that is more flexible than puts and that we need to use to implement our function: ‘print’. However, trying to use Ruby print inside the definition of Trajectories’s print will not work, as Ruby will understand that as a recursive call to print. Ruby’s print is defined inside the Kernel class, so, in order to call Ruby’s print inside the definition of Trajectories’s print we need to write ‘Kernel.print’.

```
class Trajectories

  attr_reader :times
  attr_reader :matrix

  #-----
  # Initializes the Trajectories class. Takes two parameters
  # @param times [R::Vector] should be an R vector.
  # @param matrix [R::Matrix] should be an R matrix.
  #-----

  def initialize(times: nil, matrix: nil)
    @times = times
    @matrix = matrix
  end

  #-----
  #
  #-----

  def print
    puts("*** Class Trajectories, method Print *** ")
    Kernel.print("times = ")
    puts @times
    puts("traj =")
    puts @matrix
    puts("***** End Print (trajectories) ***** ")
  end
end
```

```
@trajCochin.print
```

```
## *** Class Trajectories, method Print ***
```

```
## times = 1
## 3
## 4
## 5
## traj =
##      [,1] [,2] [,3] [,4]
## g2_v729 15.0 15.1 15.2 15.2
## g2_v730 16.0 15.9 16.0 16.4
## g2_v731 15.2  NA 15.3 15.3
## g2_v732 15.7 15.6 15.8 16.0
## ***** End Print (trajectories) *****
```

For Cochin, the result is correct. For Saint-Anne, print will display too much information. So we need a second method.

Show is the default R method used to show an object when its name is written in the console. We thus define ‘show’ by taking into account the size of the object: if there are too many trajectories, ‘show’ posts only part of them.

Here is the R code for method ‘show’:

```
> setMethod("show", "Trajectories",
+ function(object){
+ cat("*** Class Trajectories, method Show *** \n")
+ cat("* Times ="); print(object@times)
+ nrowShow <- min(10,nrow(object@traj))
+ ncolShow <- min(10,ncol(object@traj))
+ cat("* Traj (limited to a matrix 10x10) = \n")
+ print(formatC(object@traj[1:nrowShow,1:ncolShow]),quote=FALSE)
+ cat("***** End Show (trajectories) ***** \n")
+ }
+ )
```

Now, let’s write it with Galaaz. This time though, we will not rewrite the whole Trajectories class, but just reopen it to add this specific method. The next example has many interesting features of Galaaz, some we have already seen, others will be described now:

- As we have already seen, to call an R function one uses the R. notation. There is however another way: when the first argument to the R function is an R object such as a matrix, a list, a vector, etc. we can use ‘.’ notation to call the function. This makes the function look like a method of the object. For instance, `R.nrow(@matrix)`, can be called by doing `@matrix.nrow`;
- In R, every number is a length-1 vector. In Galaaz 2.0, unwrap a length-1 R vector to a Ruby number with `>> 0` (or `unboxed_get(0)`). Older Galaaz docs used `.gz / << 0` for the same idea; `<<` still works as a compatibility alias for `>>`;
- R functions and Ruby methods can be used freely in Galaaz. We show below two different ways of getting the minimum of a number, either by calling `R.min` or by getting the minimum of an array, with the `min` method;

- Galaaz allows for method ‘chaining’. Method chaining, also known as named parameter idiom, is a common syntax for invoking multiple method calls in object-oriented programming languages. Each method returns an object, allowing the calls to be chained together in a single statement without requiring variables to store the intermediate results. For instance `@matrix.nrow >> 0`, which returns the number of rows of the matrix as a Ruby number;
- Ranges in Ruby are represented by `(x..y)`, where `x` is the beginning of the range and `y` its end. An R matrix can be indexed by range, `object@traj[1:nrowShow,1:ncolShow]`, the same result is obtained in Galaaz by indexing `@matrix [(1..nrow_show), (1..ncol_show)]`. Observe that this statement is then chained with the `format` function and printed with `puts`.

```
class Trajectories

#-----
#
#-----

def show
  puts("*** Class Trajectories, method Show *** ")
  Kernel.print("times = ")
  puts @times
  nrow_show = [10, @matrix.nrow >> 0].min
  ncol_show = R.min(10, @matrix.ncol) >> 0
  puts("* Traj (limited to a matrix 10x10) = ")
  puts @matrix[(1..nrow_show), (1..ncol_show)].format(digits: 2,
    nsmall: 2)
  puts("***** End Show (trajectories) ***** ")
end

end
```

```
@trajStAnne.show
```

```
## *** Class Trajectories, method Show ***
## times = 1
## 2
## 3
## 4
## 5
## 6
## 7
## 8
## 9
## 10
## 12
## 14
## 16
## 18
## 20
## 22
## 24
```

```
## 26
## 28
## 30
## 32
## * Traj (limited to a matrix 10x10) =
##      [,1]  [,2]  [,3]  [,4]  [,5]  [,6]  [,7]  [,8]
## [1,] "16.25" "16.49" "16.28" "16.18" "16.59" "16.78" "16.93" "17.34"
## [2,] "16.00" "16.39" "16.54" "16.74" "16.74" "16.69" "16.52" "16.79"
## [3,] "16.03" "16.48" "16.21" "16.34" "16.46" "16.99" "17.07" "17.04"
## [4,] "16.18" "16.03" "16.22" "16.38" "16.74" "16.83" "16.94" "17.04"
## [5,] "16.11" "15.72" "16.01" "16.32" "16.27" "16.68" "17.18" "16.95"
## [6,] "16.03" "16.17" "16.13" "16.60" "17.02" "16.57" "17.26" "16.91"
## [7,] "15.99" "16.10" "16.51" "16.39" "16.97" "16.82" "16.74" "17.10"
## [8,] "16.32" "15.85" "16.65" "16.33" "16.60" "15.93" "16.77" "17.19"
## [9,] "16.13" "16.41" "16.56" "16.47" "16.71" "16.59" "16.94" "16.83"
## [10,] "15.77" "16.23" "16.28" "16.27" "16.83" "16.32" "16.86" "16.74"
##      [,9]  [,10]
## [1,] "17.20" "17.30"
## [2,] "17.26" "17.50"
## [3,] "17.10" "17.00"
## [4,] "17.20" "17.33"
## [5,] "17.33" "17.15"
## [6,] "17.24" "17.45"
## [7,] "17.07" "17.65"
## [8,] "16.81" "17.63"
## [9,] "17.24" "17.47"
## [10,] "16.92" "17.42"
## ***** End Show (trajectories) *****
```

Our show method has the same problem as SS4, i.e., if an empty trajectories object is created and we try to ‘show’ it, it will generate an error. Let’s see it:

```
@empty_traj = Trajectories.new
```

```
begin
  @empty_traj.show
rescue => e
  # Short message only - full backtraces overflow PDF boxes.
  puts e.class.to_s
  # Wrap long messages for PDF code-box width.
  e.message.to_s.scan(/.{1,68}/).each { |line| puts line }
end
```

```
## *** Class Trajectories, method Show ***
## times =
## NoMethodError
## undefined method 'nrow' for nil
```

In this example, @matrix is nil, so calling @matrix.nrow raises undefined method 'nrow' for nil. To fix this, we can either prevent an empty trajectories class from being created, or make sure that method show will not choke on the empty object. We will take the second

alternative, to follow SS4, and will check if either `@times` or `@matrix` is empty. If either one of them is `nil`, then we will print a message saying so.

Although the first alternative, i.e., not allow for empty objects is a possibility in Ruby, it seems that this is not the case for S4.

```
class Trajectories

  def show
    if (@times.nil? || @matrix.nil?)
      puts("*** Class Trajectories is empty!! *** ")
      return
    end
    puts("*** Class Trajectories, method Show *** ")
    Kernel.print("times = ")
    puts @times
    nrow_show = [10, @matrix.nrow >> 0].min
    ncol_show = R.min(10, @matrix.ncol) >> 0
    puts("* Traj (limited to a matrix 10x10) = ")
    puts @matrix[(1..nrow_show), (1..ncol_show)].format(digits: 2,
      nsmall: 2)
    puts("***** End Show (trajectories) ***** ")
  end
end
```

```
@empty_traj.show
```

```
## *** Class Trajectories is empty!! ***
```

11 To Remove an Object

As far as I know, there isn't a good way of removing a defined class, but there might be one and the interested user is directed to google it! In principle, there should not be any real need to remove a defined class. Both in R and Galaaz, large programs are usually written in a file and the file loaded. If one writes a wrong class, the better solution is to correct it and then load it again. If the class is written directly on the console, then leaving it there will not have any serious impact.

12 Method `count_missing`

In R, methods 'print' and 'show' are methods that already exist. SS4 wants to add a method called 'countMissing' which does not exist in R, and thus requires some special preparation. In Ruby, every method we've created is a new method that exists inside the class. The fact that 'print' happens to be also a method for class Kernel and 'show' is not, is not of special interest. Actually we've seen that in order to call method print from the Kernel class we had to call Kernel.print.

To create method 'count_missing' we just need to reopen the Trajectories class and add the method the same way we've done with method 'show'. Again, let's first look at R's 'countMissing' and then at Ruby's:

```
> setMethod(  
+ f= "countMissing",  
+ signature= "Trajectories",  
+ definition=function(object){  
+ return(sum(is.na(object@traj)))  
+ }  
+ )
```

Here we introduce another particular case of Galaaz. R has many methods that have a ‘.’ in their names, such as ‘is.na’. In Ruby, the dot ‘.’ has a special meaning as it is the way we call a method on an object. Doing ‘R.is.na’ will not work. So, in Galaaz, R functions that have a dot in them will have the dot substituted by ‘__’. So, method is.na in Galaaz, becomes R.is__na. In method count__missing we use method chaining and convert the final count to a Ruby number with >> 0 (unbox).

```
class Trajectories  
  
  def count_missing  
    return @matrix.is__na.sum >> 0  
  end  
  
end
```

```
puts @trajCochin.count_missing
```

```
## 1
```

13 To See the Methods

In order to see the methods we have defined so far, we call on class Trajectories the method ‘instance_methods’ passing it one argument, ‘false’, as follows:

```
puts Trajectories.instance_methods(false)
```

```
## count_missing  
## times  
## print  
## show  
## matrix
```

It is interesting to observe that we see our three methods ‘count_missing’, ‘print’ and ‘show’, but we also see two other methods ‘times’ and ‘matrix’, but those last two as far as we know are just instance variables and not methods, right? More on that when we talk about Accessors.

Galaaz and Ruby do not by default provide a way to see a method’s code. However, if the user uses a Ruby console such as Pry, then seeing methods and debugging is possible. Pry is beyond the scope of this document.

14 Construction

Every class in Ruby has a constructor, if not explicitly defined, at least implicitly. Method `initialize` is the constructor method and the one that coordinates the whole construction process.

15 Inspector

There is no default ‘inspector’ in Ruby as in R, although there is nothing that prevents the developer from inspecting and validating the input. For example, in the object `Trajectories`, one may want to check that the number of elements in ‘times’ is equal to the number of columns in ‘matrix’ and if they are not, issue an error. In order to understand why this restriction exists, the user is again directed to SS4.

Here we show the R code for this validation:

```
> setClass(  
+ Class="Trajectories",  
+ representation(times="numeric",traj="matrix"),  
+ validity=function(object){  
+ cat("~~~ Trajectories: inspector ~~~ \n")  
+ if(length(object@times)!=ncol(object@traj)){  
+ stop ("[Trajectories: validation] the number of temporal  
+ measurements does not correspond  
+ }else{}  
+ return(TRUE)  
+ }  
+ )
```

In order to implement this validation we will coordinate it in the `initialize` method.

```
class Trajectories  
  
  def initialize(times: nil, matrix: nil)  
    @times = times  
    @matrix = matrix  
  
    # Validate: length(@times) must equal ncol(@matrix)  
    puts ("~~~ Trajectories: inspector ~~~ ")  
    if ((@times.length >> 0) != (@matrix.ncol >> 0))  
      raise(  
        "[Trajectories: validation] the number of temporal " +  
        "measurements does not correspond with the number of " +  
        "columns in the matrix")  
    end  
  
    # show the object just created  
    show  
  
  end  
  
end
```

Let's first create a Trajectories that validates fine, i.e., the number of elements in @times is equal to the number of columns of the matrix. In this case, we will show a message saying that validation was done and then print the object.

```
ok = Trajectories.new(times: R.c(1..2), matrix: R.matrix((1..2),
  ncol: 2))
```

```
## ~~~ Trajectories: inspector ~~~
## *** Class Trajectories, method Show ***
## times = 1
## 2
## * Traj (limited to a matrix 10x10) =
## 1
## 2
## ***** End Show (trajectories) *****
```

Now, if we try to create a Trajectories that does not pass the validation criteria, our code will raise an exception. Exceptions are a standard way to deal with errors in Ruby code and many other object oriented languages. The interested reader should look for further documentation on exceptions on the web.

```
begin
  error = Trajectories.new(times: R.c(1..3), matrix: R.matrix((1..2),
    ncol: 2))
rescue => e
  # Short message only - full backtraces overflow PDF boxes.
  puts e.class.to_s
  # Wrap long messages for PDF code-box width.
  e.message.to_s.scan(/.{1,68}/).each { |line| puts line }
end
```

```
## ~~~ Trajectories: inspector ~~~
## RuntimeError
## [Trajectories: validation] the number of temporal measurements does
## not correspond with the number of columns in the matrix
```

The validation above does not consider the case when an empty object is created. Here we will check to see if either times or matrix are nil; if either one of them is nil, then we will raise an exception and interrupt the creation of the object. We also create a method validate that is called from our initialize method.

Method validate has some interesting features about the integration of Galaaz and R. We compare lengths after unboxing with `>> 0`, so the comparison is ordinary Ruby arithmetic on numbers. (A length-1 R logical can likewise be treated as a Ruby boolean via `>> 0`.)

```
class Trajectories

  def initialize(times: nil, matrix: nil)
    @times = times
    @matrix = matrix

    # call method validate to validate our input
  end
end
```

```
validate

  # show the object just created
  show

end

def validate

  # Let's first check that we do not have an empty object
  if (@times.nil? || @matrix.nil?)
    raise "Neither times nor matrix can be an empty object"
  end

  # Validate: length(@times) must equal ncol(@matrix)
  puts ("~~~ Trajectories: inspector ~~~ ")
  if ((@times.length >> 0) != (@matrix.ncol >> 0))
    raise(
      "[Trajectories: validation] the number of temporal " +
      "measurements does not correspond with the number of " +
      "columns in the matrix")
  end

end

end
```

Note: with this stricter `validate`, `Trajectories.new` no longer accepts empty objects. The earlier `@empty_traj = Trajectories.new / @empty_traj.show` pattern from part 1 no longer applies for *new* constructions; existing instances created before this reopen still exist in memory, but calling `new` with missing `times` or `matrix` will raise.

Let's try then creating an empty object:

```
begin
  error = Trajectories.new
rescue => e
  # Short message only - full backtraces overflow PDF boxes.
  puts e.class.to_s
  # Wrap long messages for PDF code-box width.
  e.message.to_s.scan(/.{1,68}/).each { |line| puts line }
end
```

```
## RuntimeError
## Neither times nor matrix can be an empty object
```

Another example:

```
begin
  error = Trajectories.new(times: 1)
rescue => e
  # Short message only - full backtraces overflow PDF boxes.
  puts e.class.to_s
  # Wrap long messages for PDF code-box width.
  e.message.to_s.scan(/.{1,68}/).each { |line| puts line }
end
```

```
## RuntimeError
## Neither times nor matrix can be an empty object
```

Let's see now that the implementation is correct and that it does not raise an error on valid input:

```
ok = Trajectories.new(times: R.c(1, 2), matrix: R.matrix((1..2),
  ncol: 2))
```

```
## ~~~ Trajectories: inspector ~~~
## *** Class Trajectories, method Show ***
## times = 1
## 2
## * Traj (limited to a matrix 10x10) =
## 1
## 2
## ***** End Show (trajectories) *****
```

The 'initialize' method is called ONLY during the initial creation of the object. If any instance variable is later modified, no control is done. At this moment though, there is no way to change the value of any of our instance variables.

```
error.times = R.c(1, 2, 3)
```

The Trajectories class works for R objects and expects as input R objects. Passing R objects in all examples has been the obligation of the programmer. Galaaz, however, can also accept many Ruby values (ranges, arrays of numbers, and so on) if we convert them at the boundary. There is no `R.convert` in Galaaz 2.0; a small helper is enough: leave `nil` alone, keep objects that are already `R::Object`, and otherwise wrap with `R.c` (which accepts ranges as well as scalars and vectors). Matrices that are already R objects are kept as-is.

```
class Trajectories

  def as_r(x)
    return nil if x.nil?
    return x if x.is_a?(R::Object)
    R.c(x)
  end

  def initialize(times: nil, matrix: nil)
    @times = as_r(times)
    @matrix = as_r(matrix)

    # call method validate to validate our input
    validate

    # show the object just created
    show
  end

  def validate
```

```
# Let's first check that we do not have an empty object
if (@times.nil? || @matrix.nil?)
  raise "Neither times nor matrix can be an empty object"
end

# Validate: length(@times) must equal ncol(@matrix)
puts ("~~~ Trajectories: inspector ~~~ ")
t1 = @times.length >> 0; mc = @matrix.ncol >> 0
if (t1 != mc)
  raise(
    "[Trajectories: validation] the number of temporal " +
    "measurements #{t1} does not correspond with the number " +
    "of columns in the matrix #{mc}")
end

end

end
```

And now let's create a new Trajectories, but we will now pass a Ruby range for times:

```
ok = Trajectories.new(times: (1..2), matrix: R.matrix((1..2), ncol: 2))
```

```
## ~~~ Trajectories: inspector ~~~
## *** Class Trajectories, method Show ***
## times = 1
## 2
## * Traj (limited to a matrix 10x10) =
## 1
## 2
## ***** End Show (trajectories) *****
```

Perfect! This works fine.

(Historical note: an earlier Galaaz prototype on Renjin also demonstrated sharing storage with the MDArray gem. Those shared-store demos are not part of Galaaz 2.0 / the GNU R bridge, and are omitted here.)

16 The Initializator

As we have seen, method 'initialize' is the main object creator orchestrator. This method can be as complex as needed. So, let's get on with some improvements to our Trajectories class.

It would be rather pleasant that the columns of the matrix of the trajectories have names, the names of measurements times. In the same way, the lines could be subscripted by a number of individual.

To do this in R, one also uses method initialize:

```
> setMethod(
+ f="initialize",
+ signature="Trajectories",
```

```
+ definition=function(.Object,times,traj){
+ cat("~~~ Trajectories: initializer ~~~ \\n")
+ colnames(traj) <- paste("T",times,sep="")
+ rownames(traj) <- paste("I",1:nrow(traj),sep="")
+ .Object@traj <- traj # Assignment of the slots
+ .Object@times <- times
+ return(.Object) # return of the object
+ }
+ )
```

In R, it is possible to assign a value to the result of a function, for example `colnames(x) <- c("v1", "v2", "v3")`. In Galaaz 2.0 the same idea is expressed with ordinary Ruby setters on the R object: `@matrix.colnames = ...` and `@matrix.rownames = ....`

```
class Trajectories

  def as_r(x)
    return nil if x.nil?
    return x if x.is_a?(R::Object)
    R.c(x)
  end

  def initialize(times: nil, matrix: nil)
    @times = as_r(times)
    @matrix = as_r(matrix)

    # call method validate to validate our input
    validate

    # Add row and column names
    puts ("~~~ Trajectories: initializer ~~~ ")
    @matrix.colnames = R.paste("T", @times, sep: "")
    @matrix.rownames = R.paste("I", (1..(@matrix.nrow >> 0)), sep: "")

    # show the object just created
    show

  end

end

@traj = Trajectories.new(times: R.c(1,2,4,8), matrix: R.matrix((1..8),
  nrow: 2))
```

```
## ~~~ Trajectories: inspector ~~~
## ~~~ Trajectories: initializer ~~~
## *** Class Trajectories, method Show ***
## times = 1
## 2
## 4
## 8
## * Traj (limited to a matrix 10x10) =
##   T1  T2  T4  T8
## I1 "1" "3" "5" "7"
```

```
## I2 "2" "4" "6" "8"  
## ***** End Show (trajectories) *****
```

Note that we still call our ‘validate’ method and it is still an error to create an empty Trajectories or one in which the sizes are wrong:

```
begin  
  error = Trajectories.new(times: R.c(1, 2, 48), matrix: R.matrix((1..8),  
    nrow: 2))  
rescue => e  
  # Short message only - full backtraces overflow PDF boxes.  
  puts e.class.to_s  
  # Wrap long messages for PDF code-box width.  
  e.message.to_s.scan(/.{1,68}/).each { |line| puts line }  
end
```

```
## ~~~ Trajectories: inspector ~~~  
## RuntimeError  
## [Trajectories: validation] the number of temporal measurements 3 doe  
## s not correspond with the number of columns in the matrix 4
```

A constructor does not necessarily take the instance variable of the object as argument. For example, if we know (that is not the case in reality, but let us imagine so) that the BMI increases by 0.1 every week, we could build trajectories by providing the number of weeks and the initial weights.

First the code in R, we skip the definition of class TrajectoriesBis:

```
> setMethod ("initialize",  
+ "TrajectoriesBis",  
+ function(.Object,nbWeek,BMIinit){  
+ traj <- outer(BMIinit, 1:nbWeek,  
+   function(init, week) { return(init + 0.1 * week) })  
+ colnames(traj) <- paste("T",1:nbWeek,sep="")  
+ rownames(traj) <- paste("I",1:nrow(traj),sep="")  
+ .Object@times <- 1:nbWeek  
+ .Object@traj <- traj  
+ return(.Object)  
+ }  
+ )
```

Now, let’s make a TrajectoriesBis in Galaaz. Here again, we should point out some characteristics of our code:

- We made initialize with two positional arguments, instead of named arguments, i.e., the first argument is the number of weeks and the second bmi_init. In this case, when making a new object the position of the arguments is important and there is no way to pass the argument by name;
- R function outer was called as if a method from bmi_init using dot notation, although one could use R.outer without problem;

- Function ‘outer’ expects an R function as its 3rd argument. In order to build an R function from Galaaz, we need to pass the function definition as a string to R.eval.

```
class TrajectoriesBis

  attr_reader :times
  attr_reader :matrix

  def initialize(number_weeks, bmi_init)
    @matrix = bmi_init.outer((1..number_weeks),
                             R.eval("function(init,
                             week) {return(init + 0.1 * week)}"))
    @times = R.c((1..number_weeks))
  end

end

@traj_bis = TrajectoriesBis.new(4, R.c(16,17,15.6))
```

```
puts @traj_bis.matrix
```

```
##      [,1] [,2] [,3] [,4]
## [1,] 16.1 16.2 16.3 16.4
## [2,] 17.1 17.2 17.3 17.4
## [3,] 15.7 15.8 15.9 16.0
```

It is always possible to pass a Ruby variable into a string by interpolating it. Put the variable inside `#{...}`. As an example, let's also require the BMI increase as a parameter. (A common mistake is to escape the interpolation — writing `\#{increment}` — which leaves the characters literally in the R source and does not substitute the Ruby value. Use real interpolation:)

```
class TrajectoriesBis

  def initialize(number_weeks, bmi_init, increment)
    @matrix = bmi_init.outer((1..number_weeks),
                             R.eval("function(init,
                             week) {return(init +
                             #{increment} * week)}"))
    @times = R.c((1..number_weeks))
  end

end

@traj_bis = TrajectoriesBis.new(4, R.c(16,17,15.6), 0.3)
```

```
puts @traj_bis.matrix
```

```
##      [,1] [,2] [,3] [,4]
## [1,] 16.3 16.6 16.9 17.2
## [2,] 17.3 17.6 17.9 18.2
## [3,] 15.9 16.2 16.5 16.8
```

17 Constructors for Users

Many times, it is interesting to have different ways of constructing an object depending on what information our users have or want to provide to the constructor. Although we have only one initialize method, we can create multiple methods, that do some preprocessing and then call the initialize method to carry out the object building.

In order to do that, we use what are called class methods, instead of instance methods. All the methods we've created so far are instance methods; class methods are defined by prepending the self keyword to the method's name. Still using the assumption that the BMI will grow by 0.1 per week, let's define a regular trajectory without having to define a TrajectoriesBis as above:

```
> regularTrajectories <- function(nbWeek,BMIinit) {
+ traj <- outer(BMIinit, 1:nbWeek,
+   function(init, week) { return(init + 0.1 * week) })
+ times <- 1: nbWeek
+ return(new(Class="Trajectories",times=times,traj=traj))
+ }
> regularTrajectories(nbWeek=3,BMIinit=c(14,15,16))
```

Notice how method 'regular' is defined as 'self.regular', making it a class method. The last statement of the method definition is actually a call to the Trajectories constructor 'new' passing the calculated values for times and matrix.

Notice also how method regular is called, similar to the way new is called by adding it after class Trajectories name: 'Trajectories.regular'.

```
class Trajectories
```

```
  def self.regular(number_weeks: nil, bmi_init: nil)
    matrix = bmi_init.outer((1..number_weeks),
                           R.eval("function(init,
                                         week) {return(init + 0.1 * week)}"))
    times = R.c((1..number_weeks))
    Trajectories.new(times: times, matrix: matrix)
  end
```

```
end
```

```
@regular = Trajectories.regular(bmi_init: R.c(14, 15, 16),
  number_weeks: 3)
```

```
## ~~~ Trajectories: inspector ~~~
## ~~~ Trajectories: initializer ~~~
## *** Class Trajectories, method Show ***
## times = 1
## 2
## 3
## * Traj (limited to a matrix 10x10) =
##   T1      T2      T3
## I1 "14.10" "14.20" "14.30"
```

```
## I2 "15.10" "15.20" "15.30"
## I3 "16.10" "16.20" "16.30"
## ***** End Show (trajectories) *****
```

We have already seen that constructors can be as complex as needed, calling other methods and doing calculations on the received parameters. On this last example, we will check if the times variable was provided. If it is not provided, then we will use matrix columns to define the times:

```
class Trajectories

  def self.init(times: nil, matrix: nil)
    times = R.c((1..(matrix.ncol >> 0))) if times.nil?
    Trajectories.new(times: times, matrix: matrix)
  end

end

@traj = Trajectories.init(matrix: R.matrix((1..8), ncol: 4))

## ~~~ Trajectories: inspector ~~~
## ~~~ Trajectories: initializer ~~~
## *** Class Trajectories, method Show ***
## times = 1
## 2
## 3
## 4
## * Traj (limited to a matrix 10x10) =
##   T1 T2 T3 T4
## I1 "1" "3" "5" "7"
## I2 "2" "4" "6" "8"
## ***** End Show (trajectories) *****
```

18 Accessors

Accessors are methods for getting and setting the value of instance variables.

19 Get

Getters are methods for getting the value of an instance variable. We have been using getters since the beginning of this document, without explicitly saying so. When defining `attr_reader :times` and `attr_reader :matrix`, we have actually defined two getter methods for reading the values of variables `times` and `matrix` respectively. We can however define getters explicitly:

```
class TrajectoriesBis

  def initialize(times: nil, matrix: nil)
    @times = times
    @matrix = matrix
  end

end
```

```
end

def times
  @times
end

def matrix
  @matrix
end

end

@traj = TrajectoriesBis.new(times: 1, matrix: 2)
```

```
puts @traj.times
```

```
## 1
```

```
puts @traj.matrix
```

```
## 2
```

It is also possible to define more sophisticated getters. For example one can regularly need the BMI at inclusion. In R, one would index a matrix as `matrix[,1]`. In Ruby, it is a syntax error to have a `,` just after the `[`. In this case we need to add `'nil'` as in `matrix[nil, 1]`:

```
class Trajectories

  def get_traj_inclusion
    @matrix[nil, 1]
  end

end
```

```
puts @trajCochin.get_traj_inclusion
```

```
## numeric(0)
```

20 Set

A setter is a method that assigns a value to a variable. As with getters, Ruby also provides an easy way to write setters and allow you to also write them explicitly. Let's first use the simple way:

```
class TrajectoriesBis

  attr_writer :times
  attr_writer :matrix

  def initialize(times: nil, matrix: nil)
```

```
@times = times
@matrix = matrix
end

end

@traj = TrajectoriesBis.new
@traj.times = R.c(1, 2)
@traj.matrix = R.matrix((1..2), ncol: 2)

puts @traj.matrix
```

```
##      [,1] [,2]
## [1,]    1    2
```

Note that now we can use '=' to assign a value to both variables times and matrix. Without setters, changing the value of variables times and matrix was not possible. Our class, up to this point was protected from any changes to those variables. If we need to allow changes to those variables, then setters are needed. In this case, the simple setter as shown above is not ideal, since it would allow changes that break the restriction that variable times has to have the same length as the number of columns of matrix. In order to do the verification we need to implement a more sophisticated setter. In the example below, we add the 'times=' setter that receives as input one argument. First we convert the given argument to an R object, then check to see that the length of times is the same as the number of columns and if everything is fine, then we set the value of instance variable times:

```
class Trajectories

  def as_r(x)
    return nil if x.nil?
    return x if x.is_a?(R::Object)
    R.c(x)
  end

  def times=(times)
    times = as_r(times)
    tl = times.length >> 0; mc = @matrix.ncol >> 0
    if (tl != mc)
      raise(
        "[Trajectories: validation] the number of temporal " +
        "measurements #{tl} does not correspond with the number " +
        "of columns in the matrix #{mc}")
    end
    @times = times
  end

end

begin
  @trajCochin.times = (1..5)
rescue => e
  # Short message only - full backtraces overflow PDF boxes.
  puts e.class.to_s
  # Wrap long messages for PDF code-box width.
end
```

```
e.message.to_s.scan(/.{1,68}/).each { |line| puts line }  
end
```

```
## RuntimeError  
## [Trajectories: validation] the number of temporal measurements 5 doe  
## s not correspond with the number of columns in the matrix 4
```

We now set the value appropriately and will not get any errors:

```
@trajCochin.times = R.c(1, 5, 6, 8)
```

21 The Operator '['

It is also possible to define getters by using the operator '['. This operator is not usually used for returning instance variables and it is preferable to use the methods we've used above; however, for completeness with SS4 we are showing how to define this here. Operator '[' is better left to be used for array/matrix indices.

```
class Trajectories  
  
  def [] (var_name)  
  
    case var_name  
    when "times"  
      @times  
    when "matrix"  
      @matrix  
    else  
      raise "Unknown instance variable"  
    end  
  
  end  
  
end
```

```
puts @trajCochin["times"]
```

```
## [1] 1 5 6 8
```

Similarly, we could use operator '['=' to assign a value to times and matrix. We will not do this here as we think that the other options are better and the interested user can easily find help, if needed to implement such method.

22 To Go Further

This section will introduce advanced features of Object Oriented programming such as Inheritance and Modules and will also show some aspects of S4 that do not apply to Ruby.

23 Methods Using Several Arguments

In Ruby, methods can have as many arguments as needed and those methods are defined the way we have already seen in many of the examples above. The example in SS4 presents a method that prints different output if its input is numeric, character or both. Let's write a class in Ruby that does the same for Numeric and String. In Ruby we do not define global functions, we always define methods inside classes or modules (as we will see later). Also, Ruby is not typed, so methods are not called depending on their types as in SS4 examples. Below, method test will be called with one parameter. At the time of calling we do not know the type of the argument; the method can then check if the received argument is a Numeric or a String and at this time, decide what should be printed.

```
class Test

  def test(input)

    case input
    when Numeric
      puts "The input is numeric: #{input}"
    when String
      puts "The input is a string: #{input}"
    else
      puts "The input is neither a number nor a string"
    end
  end

end

@t = Test.new
```

```
@t.test(5)
```

```
## The input is numeric: 5
```

```
@t.test("Hello")
```

```
## The input is a string: Hello
```

Ruby has ways of dealing with multiple arguments, missing arguments, undefined number of arguments, named arguments, unnamed arguments, etc. This is beyond the scope of this document and we suggest the interested reader to go to the many resources about Ruby that can easily be found on the web.

We will now create a new class 'Partition' that we will use later in this document. This class will have only the basic methods needed for the examples to work.

```
class Partition

  attr_reader :nb_groups
  attr_reader :part
```

```
def initialize(nb_groups, part)
  @nb_groups = nb_groups
  @part = part
end

end

@partCochin = Partition.new(2, R.c("A","B","A","B").factor)
@partStAnne = Partition.new(2, R.c("A","B").rep(R.c(50,30)).factor)
```

```
puts @partCochin.part
```

```
## [1] A B A B
## Levels: A B
```

```
puts @partStAnne.part
```

```
## [1] A A A A A A A A A A A A A A A A A A A A A A A A A A A A A
## [34] A A A A A A A A A A A A A A A A B B B B B B B B B B B B B B B
## [67] B B B B B B B B B B B B B B B
## Levels: A B
```

We will suppose that part is always composed of capital letters going from A to LETTERS[nb_groups].

24 Inheritance

Ruby being a powerful Object Oriented language has the concept of Inheritance, but it does not allow for multiple inheritance. Multiple inheritance has many drawbacks and Ruby just does not support it. However, Ruby has other concepts that make up for the lack of multiple inheritance as we will see in the following examples.

So, let's go back to SS4 examples. We want now to define a class called TrajPartitioned that inherits from class Trajectories. When a class has a parent, all methods available for the parent are also available to the child.

```
class TrajPartitioned < Trajectories

  attr_reader :list_partitions

end
```

That's all there is to it! We've just created a class TrajPartitioned that inherits all methods from class Trajectories and at this point does nothing different from Trajectories, but adds a new instance variable: list_partitions.

Creating TrajPartitioned without arguments will generate an error, since a Trajectories requires both times and matrix to be non null.

```
begin
  @tdPitie = TrajPartitioned.new
rescue => e
  # Short message only - full backtraces overflow PDF boxes.
  puts e.class.to_s
  # Wrap long messages for PDF code-box width.
  e.message.to_s.scan(/.{1,68}/).each { |line| puts line }
end
```

```
## RuntimeError
## Neither times nor matrix can be an empty object
```

Let's try to create a TrajPartitioned, but passing to it two partitions. For that, let's first create a new Partition:

```
@partCochin2 = Partition.new(3, R.c("A", "C", "C", "B").factor)
```

And now let's create the TrajPartitioned:

```
begin
  @tdCochin = TrajPartitioned.new(times: R.c(1,3,4,5),
    matrix: @trajCochin.matrix,
                                list_partitions: R.list(@partCochin,
                                                         @partCochin2))
rescue => e
  # Short message only - full backtraces overflow PDF boxes.
  puts e.class.to_s
  # Wrap long messages for PDF code-box width.
  e.message.to_s.scan(/.{1,68}/).each { |line| puts line }
end
```

```
## ArgumentError
## unknown keyword: :list_partitions
```

This didn't work: R function 'list' expects R objects, and in this case, @partCochin and @partCochin2 are Ruby classes, so trying to apply function list to them does not work. Clearly, we will have to work in the realm of Ruby to keep the list of partitions. This is not a problem as Ruby has data structures to maintain a list of objects, the Array. Let's then try another solution:

```
begin
  @tdCochin = TrajPartitioned.new(times: R.c(1,3,4,5),
    matrix: @trajCochin.matrix,
                                list_partitions: [@partCochin,
                                                  @partCochin2])
rescue => e
  # Short message only - full backtraces overflow PDF boxes.
  puts e.class.to_s
  # Wrap long messages for PDF code-box width.
  e.message.to_s.scan(/.{1,68}/).each { |line| puts line }
end
```

```
## ArgumentError
## unknown keyword: :list_partitions
```

We now get a second error: ‘unknown keyword: list_partitions’. Class TrajPartitioned inherits from class Trajectories and class Trajectories has an initialize function that requires two parameters, times and matrix; list_partitions is not a parameter for initialize and is thus unknown. In order to fix this problem we need to create an initialize method for class TrajPartitioned.

25 The ‘super’ Keyword

R has a method called ‘callNextMethod’ for control flow between inherited classes. In Ruby, we have a model that is a bit different. When a method is called on a subclass, if this method is not found it will be searched in the parent class and it will go up the hierarchy of classes until it is found or an error is issued. If we want the parent method to be called we can call ‘super’:

```
class TrajPartitioned

  def initialize(times: nil, matrix: nil, list_partitions: nil)
    super(times: times, matrix: matrix)
    @list_partitions = list_partitions
  end

end
```

Let’s try our example again:

```
@tdCochin = TrajPartitioned.new(times: R.c(1,3,4,5),
                                matrix: @trajCochin.matrix,
                                list_partitions: [@partCochin,
                                                  @partCochin2])
```

```
## ~~~ Trajectories: inspector ~~~
## ~~~ Trajectories: initializer ~~~
## *** Class Trajectories, method Show ***
## times = 1
## 3
## 4
## 5
## * Traj (limited to a matrix 10x10) =
##   T1      T3      T4      T5
## I1 "15.00" "15.10" "15.20" "15.20"
## I2 "16.00" "15.90" "16.00" "16.40"
## I3 "15.20" "   NA" "15.30" "15.30"
## I4 "15.70" "15.60" "15.80" "16.00"
## ***** End Show (trajectories) *****
```

Now @tdCochin is created correctly; however, the ‘show’ method only shows information about times and matrix, there is nothing about our new list_partitions variable. This is so, since there is no method ‘show’ in TrajPartitioned, so method ‘show’ from Trajectories is executed.

So, let's start by writing a 'print' method, that will print all the information we have in `TrajPartitioned`. The flow of control for this method is: Ruby sees a call to 'print', so it checks to see if 'print' is a method for `TrajPartitioned`. Since we have just defined this method, Ruby finds it and executes it. The first command in print is a call to 'super', which will call the parent 'print' method, that prints information for 'times' and 'matrix'. When the parent 'print' finishes control continues after the 'super' call, printing the number of available partitions.

```
class TrajPartitioned

  def print
    super
    puts(
      "the object also contains " +
      "#{@list_partitions.length} partition")
    puts ("***** Fine of print (TrajPartitioned) *****")
  end
end
```

```
@tdCochin.print
```

```
## *** Class Trajectories, method Print ***
## times = 1
## 3
## 4
## 5
## traj =
##      T1      T3      T4      T5
## I1 15.0 15.1 15.2 15.2
## I2 16.0 15.9 16.0 16.4
## I3 15.2   NA 15.3 15.3
## I4 15.7 15.6 15.8 16.0
## ***** End Print (trajectories) *****
## the object also contains 2 partition
## ***** Fine of print (TrajPartitioned) *****
```

Notice that this model is much cleaner than 'callNextMethod' and is not subject to any of the difficulties presented in SS4 and there is no need for the keywords "is", "as" and "as<=", although Ruby provides methods to check the class of an object, its hierarchy, etc. when needed.

In Ruby there is no similar method as "setIs" and it is not possible to convert one class into another, but there are other ways of getting the necessary results. Let's then implement a method that returns the partition with the least number of groups. First, as usual, the R code with 'setIs':

```
> setIs(
+ class1="TrajPartitioned",
+ class2="Partition",
+ coerce=function(from,to){
+   numberGroups <- sapply(tdCochin@listPartitions,getNbGroups)
+   Smallest <- which.min(-numberGroups)
+   to<-new("Partition")
```

```
+ to@nbGroups <- getNbGroups(from@listPartitions[[Smallest]])
+ to@part <- getPart(from@listPartitions[[Smallest]])
+ return(to)
+ }
+ )
```

And now the Ruby code. Here we are getting deeper into Ruby and it is becoming harder for a pure R developer to understand the code. We will describe it in more detail:

- We define a method called 'to_part' that has one argument 'which'. By default 'which' is ':min', the name of the minimum method. This means that if no argument is given to to_part it will assume which = :min;
- @list_partitions is a Ruby array. Method map is similar to method supply in R, it applies a 'block' to every element of the array, returning an array. Describing blocks is beyond the scope of this document, but we can think of it as if it were a function. The block is in '{ }' and has one argument named 'part'. Thus, map goes through all elements of the array, and gets the nb_groups of the element and returns them into the number_groups array.
- number_groups is an array and doing number_groups.min returns the minimum value in number_groups and number_groups.max the maximum. We can call a method on an object by 'sending' the method name to the object, so, number_groups.send(:min) is equivalent to number_groups.min;
- Method 'index' for array, returns the index of a given element. So, number_groups.index(3) would return the index of the element '3'. Then number_groups.index(number_groups.min) returns the index of the minimum element in the array. This is the equivalent of R which.min(number_groups);
- Finally, number_groups.index(number_groups.send(which)), will return the index of the element we ask for, be it :min or :max. Note that if we pass another value, this would be an error.

```
class TrajPartitioned

  def to_part(which = :min)
    number_groups = @list_partitions.map { |part| part.nb_groups }
    selected = number_groups.index(number_groups.send(which))
    return @list_partitions[selected]
  end
end
```

To get the partition with the minimum number of elements:

```
puts @tdCochin.to_part.part
```

```
## [1] A B A B
## Levels: A B
```

To get the partition with the maximum number of elements:

```
puts @tdCochin.to_part(:max).part
```

```
## [1] A C C B  
## Levels: A B C
```

In this example we did not follow exactly the R code from SS4. The reason for that is that ‘list_partitions’ is a list of Ruby classes and we cannot run `sapply` on this list. If we try to call a ‘getNbGroups’ or in the Ruby case `nb_groups` via R’s `sapply`, the code will crash.

26 Virtual Classes

In Ruby there are no “Virtual Classes”, but it is possible to implement derived classes from a parent class with methods that behave properly according to the object’s class. Following SS4 we will implement two classes: `PartitionSimple` and `PartitionEval` which are subclasses of class `PartitionFather`. `PartitionFather` will just be a regular class. Methods defined in `PartitionFather` will be available to be used in the subclasses

Here is the R code of those classes and the implementation of a method in `PartitionFather` that multiplies the number of groups by 2:

```
> setClass(  
+ Class="PartitionFather",  
+ representation=representation(nbGroups="numeric","VIRTUAL")  
+ )  
  
> setClass(  
+ Class="PartitionSimple",  
+ representation=representation(part="factor"),  
+ contains="PartitionFather"  
+ )  
  
> setClass(  
+ Class="PartitionEval",  
+ representation=representation(part="ordered"),  
+ contains="PartitionFather"  
+ )  
  
> setGeneric("nbMultTwo",function(object){standardGeneric("nbMultTwo")})  
  
> setMethod("nbMultTwo","PartitionFather",  
+ function(object){  
+ object@nbGroups <- object@nbGroups*2  
+ return (object)  
+ }  
+ )
```

Since Ruby has no type definition, there is no really need for a parent class and subclasses. However, we will implement those classes in order to show Ruby’s inheritance:

```
# Parent class. Differently from SS4, both 'nb_groups' and 'part' are
# defined in the
# parent class.
class PartitionFather

  attr_reader :nb_groups
  attr_reader :part

  # initialize class PartitionFather with the number of groups and
  # parts. Note that we
  # use R.c for nb_groups in order to convert the number of groups into
  # an R vector.
  def initialize(nb_groups: 0, part: nil)
    @nb_groups = R.c(nb_groups)
    @part = part
  end

  # method nb_mult_two can be called from all subclasses
  def nb_mult_two
    @nb_groups * 2
  end

  # method 'to_s' is called whenever we try to print a Ruby object.
  # This method emulates
  # R 'print' method that prints all the slots.
  def to_s
    puts ("Variable 'nb_groups':")
    puts @nb_groups
    puts
    puts ("Variable 'part':")
    puts @part
    puts
  end

end

# Class PartitionSimple is a subclass of PartitionFather. To make a
# subclass of a
# class we use the operator '<'. Since the whole logic is in the parent
# class
# PartitionSimple is just an empty class
class PartitionSimple < PartitionFather

end

# PartitionEval is also only an empty class
class PartitionEval < PartitionFather

end

@a = PartitionSimple.new(
  nb_groups: 3,
  part: ((~R[:LETTERS])[R.c(1, 2, 3, 2, 2, 1)].factor))
puts @a
```

```
## Variable 'nb_groups':
```



```
## 3
##
## Variable 'part':
## [1] A B C B B A
## Levels: A B C
##
## #<RC::PartitionSimple:0x6b85a357>
```

```
puts @a.nb_mult_two
```

```
## [1] 6
```

```
@b = PartitionEval.new(
  nb_groups: 5,
  part: (~R[:LETTERS])[R.c(1, 5, 3, 4, 2, 4)].ordered)
puts @b
```

```
## Variable 'nb_groups':
## 5
##
## Variable 'part':
## [1] A E C D B D
## Levels: A < B < C < D < E
##
## #<RC::PartitionEval:0x462891eb>
```

```
puts @b.nb_mult_two
```

```
## [1] 10
```

The example above, although it replicates SS4 is not actually very useful from the point of view of class hierarchy in Ruby. We will then write a new function `to_s` in class `PartitionSimple` that will print the name of the class:

```
class PartitionSimple

  def to_s
    puts("Class PartitionSimple")
    super
  end

end
```

```
puts @a
```

```
## Class PartitionSimple
## Variable 'nb_groups':
## 3
##
```

```
## Variable 'part':  
## [1] A B C B B A  
## Levels: A B C  
##  
## #<RC::PartitionSimple:0x6b85a357>
```

As can be seen, 'puts @a' now calls method 'to_s' defined in class PartitionSimple. This method prints 'Class PartitionSimple' and then calls the super method, i.e., method 'to_s' from class PartitionFather.

Note though that 'puts @b' still prints the same output, since it has no particular 'to_s' method.

```
puts @b
```

```
## Variable 'nb_groups':  
## 5  
##  
## Variable 'part':  
## [1] A E C D B D  
## Levels: A < B < C < D < E  
##  
## #<RC::PartitionEval:0x462891eb>
```

27 Internal Modification of an Object

27.1 Method to Modify a Field

Let us return to our trajectories example and define a third method that imputes data for missing values. To simplify, we will impute by replacing by the mean values. This is the R code to do this:

```
> meanWithoutNa <- function (x){mean(x,na.rm=TRUE)}  
> setGeneric("impute",function (.Object){standardGeneric("impute")})  
> setMethod(  
+ f="impute",  
+ signature="Trajectories",  
+ def=function(.Object){  
+ average <- apply(.Object@traj,2,meanWithoutNa)  
+ for (iCol in 1:ncol(.Object@traj)){  
+ .Object@traj[is.na(.Object@traj[,iCol]),iCol] <- average[iCol]  
+ }  
+ return(.Object)  
+ }  
+ )
```

The code above, as explained in SS4 creates a new object and does not change the original one. So, calling `impute(trajCochin)` will work correctly by creating a new object but will not change `trajCochin`. This works fine, but can be memory expensive if the matrix is a large one.

Let's now implement the same method in Galaaz 2.0. We stay on the R side of the bridge: for each column, compute the mean with `na.rm = true`, then replace NA entries with that mean (via `R.ifelse / is_na`), and rebuild the matrix with `R.cbind`. No MDArray iteration is required.

```
class Trajectories

  def impute
    ncols = @matrix.ncol >> 0
    imputed = (1..ncols).map do |j|
      col = @matrix[nil, j]
      avg = col.mean(na_rm: true)
      R.ifelse(col.is_na, avg, col)
    end
    col_names = @matrix.colnames
    row_names = @matrix.rownames
    @matrix = R.cbind(*imputed)
    @matrix.colnames = col_names unless col_names.nil?
    @matrix.rownames = row_names unless row_names.nil?
    self
  end

end
```

```
@trajCochin.impute
puts @trajCochin.matrix
```

```
## $rownames
## [1] "I1" "I2" "I3" "I4"
```

It works, and `@trajCochin.matrix` was updated. Under GNU R, assignment follows R's usual copy-on-write semantics: replacing `@matrix` (or assigning into an R object through the bridge) binds a new vector/matrix rather than mutating a shared MDArray store. That is a deliberate difference from the Renjin/MDArray mutation experiments in the older paper; those demos are not part of Galaaz 2.0.

28 Conclusions I

This ends the SS4 paper material for classes and inheritance. We believe we have shown that R S4 can be substituted by Galaaz and Ruby classes and that Galaaz makes an easy transition from R developers to Ruby. Ruby is a very flexible and powerful language and has many interesting libraries, where Rails is maybe one of the best known, but there are thousands of others. For those interested in getting deeper into Ruby's libraries, we suggest they look at:

- <https://github.com/markets/awesome-ruby>
- <http://bestgems.org/>

For those interested in Ruby and science, we recommend:

- <http://sciruby.com/>

Galaaz 2.0 runs on **JRuby** or **CRuby** and talks to **GNU R** through the bridge described in this series — the same integration model used throughout the examples above.

29 Callbacks and R calling into Ruby

On this paper we have focused on accessing R functions from Ruby and have shown how to integrate Ruby with R from the point of view of a Ruby developer. The complementary direction — R calling back into Ruby — is also supported in Galaaz 2.0.

Galaaz 2.0 uses the **bridge callback** mechanism: Ruby procs (and related callables) can be passed where R expects functions, so algorithms written in R (for example optimizers or higher-order ***apply** helpers) can invoke Ruby logic without leaving the bridge session. Details, options such as callback timeouts, and further examples are in the project manual and on the documentation site: <https://rbotafo.github.io/galaaz/>.

We do not reproduce here the older Renjin-era material on packing Ruby objects as R external pointers, constructing Ruby classes from R via JVM APIs, or calling Java collections from R scripts. Those sections belonged to a different runtime; the callback bridge is the supported path in Galaaz 2.0.

30 Conclusions II

JRuby or CRuby + GNU R + Galaaz gives a practical polyglot stack: idiomatic Ruby for structure and libraries, GNU R for statistics and the CRAN/Bioconductor ecosystem, and Galaaz as the bridge between them. As always, choose the right tools for the job at hand — and when the job sits between an R-only workflow and a broader polyglot application, Galaaz is designed to connect those worlds.